Optimizing Automatic Differentiation in Julia: Comparison with Pytorch and Flux

1st Rafał Pyłko
Department of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
rafal.pylko.stud@pw.edu.pl

2nd Michał Jagodziński
Department of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
michal.jagodzinski2.stud@pw.edu.pl

***Abstract*—This article presents a custom AD engine built from scratch in Julia, implementing backward accumulation via a computational graph. To achieve competitive performance, we applied several low-level optimizations: im2col transformations for convolutional layers, strict type stability enforcement, Float32 precision, and memory-efficient in-place operations combined with loop vectorization (@simd) and bounds checking elimination (@inbounds). We benchmarked our optimized library against PyTorch and Flux on a standard CNN architecture with fulll FashionMNIST dataset. Results show that our optimized implementation completes three training epochs in 19 seconds, compared to 30 seconds for Flux and 35 seconds for PyTorch. Additionally, our engine's memory allocation is only 30% of that required by Flux. This study demonstrates that Julia can deliver exceptional performance for custom deep learning tools.**

***Index Terms*—Julia, CNN, optimization, PyTorch, Neural Networks, SDG, Auto Differentiation**

I. Introduction

Deep learning architectures, particularly Convolutional Neural Networks (CNNs), demand massive computational resources and highly efficient gradient computation. Automatic Differentiation (AD) serves as the mathematical foundation of this process, enabling the dynamic and accurate computation of derivatives necessary for backpropagation. While established frameworks like PyTorch and Flux provide highly optimized, production-ready AD engines, their heavily abstracted nature often obscures the underlying performance bottlenecks and memory management challenges.

The Julia programming language presents a compelling environment for deep learning by solving the "two-language problem" – promising high-level, mathematically intuitive syntax combined with C-like execution speed. However, because Julia is dynamically typed, realizing its full performance potential requires deliberate architectural decisions. Naive implementations of neural network training in Julia frequently suffer from type instability, excessive memory allocation, and unoptimized loops. These issues invoke heavy Garbage Collector overhead and drastically degrade execution speed.

To investigate and overcome these fundamental bottlenecks, we developed a custom AD library from scratch, implementing backward accumulation over a dynamic computational graph. Building a custom engine provides granular control over the entire execution pipeline. This paper explores the critical optimizations required to make a custom Julia AD engine highly performant. By applying techniques such as im2col transformations, Float32 precision enforcement, strict type stability, and memory-efficient in-place operations, we demonstrate how to effectively bypass the overhead.

II. Literature review

III. Methodology and implementation

• opis wybranego podejścia (różniczkowanie w przód/wstecz, graf obliczeniowy/metaprogramowanie etc.) • struktura biblioteki • wyzwania zwiazane z implementacja (o charakterze naukowym!) • opis wykonanych optymalizacji (w tym zwiazanych ze sposobem implementacji)

A. Types instability

B. im2col

C. float32

D. Weights initialization

E. In-place operations

IV. Comparison with other libraries

A. Time complexity

V. Test and comparison

TABLE I
My Network efficiency results

Trial	Time (s)	Allocs	Mem (GB)	Train Acc (%)	Test Acc (%)
1	21,74	$4,24 \times 10^7$	2,47	87,74	86,53
2	21,99	$4,24 \times 10^7$	2,47	87,78	86,53
3	22,02	$4,24 \times 10^7$	2,47	87,87	86,67
4	21,50	$4,24 \times 10^7$	2,47	87,74	86,62
5	21,81	$4,24 \times 10^7$	2,47	87,64	86,57

TABLE II
Flux efficiency results

Trial	Time (s)	Allocs	Mem (GB)	Train Acc (%)	Test Acc (%)
1	42,47	$6,01 \times 10^7$	26,09	88,10	86,88
2	29,12	$1,25 \times 10^7$	23,83	88,04	86,65
3	28,96	$1,25 \times 10^7$	23,83	87,44	86,50
4	26,76	$1,25 \times 10^7$	23,83	87,14	85,98
5	27,62	$1,25 \times 10^7$	23,83	87,94	86,52

TABLE III
PyTorch efficiency results

Trial	Time (s)	Train Acc (%)	Test Acc (%)
1	32.94	86.93	85.75
2	32.52	87.74	86.79
3	32.47	87.47	86.01
4	32.21	87.56	86.00
5	33.09	87.64	86.66

A. Memory usage

VI. Conclusion

Julia is fast.

References

[1] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," Phil. Trans. Roy. Soc. London, vol. A247, pp. 529–551, April 1955.